

Propuesta de Proyecto

Descubrimiento de ecuaciones para Gemelos Digitales
mediante un enfoque híbrido de IA aceleradora y HPC

José Sebastian Sasías, MSc. Ing.

PEDECIBA Informática

Facultad de Ingeniería - Universidad de la República, Uruguay

Junio, 2026

Resumen

La construcción de gemelos digitales requiere modelos matemáticos precisos y adecuados para una implementación computacional. El objetivo de este trabajo es descubrir automáticamente ecuaciones diferenciales interpretables a partir de datos de calidad de entrada/salida de dispositivos físicos, permitiendo la construcción de Gemelos Digitales transparentes y fiables. Se definen aspectos fundamentales de una propuesta donde se consideran objetivos, alcance incremental, fuentes de datos abiertas (SRBench, cp3-bench), marco teórico, metodología ajustada a la propuesta, cronograma acorde y una visión sobre resultados esperados.

Este proyecto propone un enfoque híbrido innovador que combina:

- **Técnicas de Inteligencia Artificial (IA):** redes neuronales de grafos (GNN) como predictores de fitness, autoencoders variacionales (VAE) para generación inteligente de modelos, y aprendizaje por refuerzo (RL) para adaptación dinámica de la búsqueda.
- **Computación de Altas Prestaciones (HPC):** ejecución paralela de simulaciones costosas, con balance de carga y tolerancia a fallos.
- **Optimización paramétrica estructurada:** exploración de familias de modelos predefinidas, sin descartar la posibilidad de uso de regresión simbólica (condicionado a existencia de herramientas maduras disponibles).

La IA actúa como **acelerador inteligente**, filtrando un alto porcentaje de las evaluaciones costosas y guiando la búsqueda hacia regiones prometedoras. Se espera reducir el tiempo de descubrimiento de semanas a horas mediante uso de HPC.

Índice

1. Descripción del problema	3
1.1. Formulación matemática	3
1.2. Familias de modelos consideradas	3
1.3. Rol de la Inteligencia Artificial	3
2. Justificación del uso de HPC	4
2.1. Dimensionalidad del espacio de búsqueda	4
2.2. Costo computacional de la evaluación	4
2.3. Paralelización natural	4
2.4. Estimación de recursos	4
3. Estrategia de resolución	5
3.1. Arquitectura general del sistema	5
3.2. Métodos de optimización de parámetros	5
3.3. Balance de carga	5
3.4. Tolerancia a fallos	5
3.5. Aceleración esperada por IA	6
4. Propuesta de evaluación experimental	6
4.1. Métricas de desempeño computacional	6
4.2. Métricas de calidad del modelo	6
4.3. Experimentos planificados	7
5. Resumen de la propuesta	7
Apéndice A: Glosario de Términos	8
Referencias Bibliográficas	10

1. Descripción del problema

1.1. Formulación matemática

Dado un sistema dinámico con:

- Señales de entrada: $\mathbf{u}(t) \in \mathbb{R}^m$
- Señales de salida: $\mathbf{y}(t) \in \mathbb{R}^p$

Se busca una función $\mathcal{F}$ y parámetros $\boldsymbol{\theta}$ tales que:

$$\mathcal{F}[\mathbf{y}(t), \dot{\mathbf{y}}(t), \ddot{\mathbf{y}}(t), \dots, \mathbf{u}(t), \boldsymbol{\theta}] = 0 \quad \forall t$$

Esta aproximación genérica es factible, ya que el foco de interés se encuentra en sistemas mecatrónicos que son utilizados en industrias, de modo tal que todos ellos NO son invariantes en el tiempo, por lo que necesariamente existe una vinculación operativa de las señales de salida con sus derivadas respecto al tiempo.

1.2. Familias de modelos consideradas

Se han considerado familias de modelos que típicamente permiten modelar sistemas mecánicos, neumáticos, hidráulicos, térmicos, eléctricos y electrónicos.

Cuadro 1: Familias de modelos paramétricos

Familia	Expresión general	Parámetros
Lineal	$\sum_k A_k y^{(k)} = \sum_k B_k u^{(k)}$	Coeficientes A_k, B_k
Polinómica	$\sum \alpha_{ij} y^i \dot{y}^j = \sum \beta_k u^k$	Coeficientes α_{ij}, β_k
Racional	$P(y, \dot{y})/Q(y, \dot{y}) = R(u)$	Coeficientes de P, Q, R
Exponencial	$y(t) = A \cdot e^{\alpha \cdot u(t)}$	Coeficientes de A, α
Fourier	$y(t) = \sum_k [a_k \cos(k\omega t) + b_k \sin(k\omega t)]$	Amplitudes a_k, b_k , frecuencia ω

Pudiendo considerarse también combinaciones lineales entre estos tipos o familias.

1.3. Rol de la Inteligencia Artificial

- **Predictor GNN:** Estima el error de modelos candidatos sin simulación costosa, en este tipo de problemas es usual un nivel de filtrando entre el 80-90 % de las evaluaciones.
- **VAE generativo:** Aprende un espacio latente de expresiones matemáticas para generar mutaciones inteligentes.
- **Agente RL (PPO):** Ajusta dinámicamente hiperparámetros de búsqueda (tasa de mutación, presión selectiva, etc.).

2. Justificación del uso de HPC

2.1. Dimensionalidad del espacio de búsqueda

Cuadro 2: Dimensionalidad del espacio de búsqueda

Familia	Parámetros (p)	Valores/rango (r)	Combinaciones
Lineal orden 2	3	100	10^6
Lineal orden 5	6	100	10^{12}
Polinómica grado 3	10	50	10^{17}

2.2. Costo computacional de la evaluación

Cada evaluación de un conjunto de parámetros requiere integración numérica de EDOs ($O(N \cdot n^3)$) y validación cruzada (k veces).

2.3. Paralelización natural

La evaluación de diferentes estructuras y parámetros es **forzosamente paralela**:

- Cada worker evalúa una combinación independiente.
- Comunicación mínima (solo al inicio y al final).
- GPUs para simulaciones intensivas.

2.4. Estimación de recursos

En esta sección, se muestran algunas consideraciones y datos de referencias representativas tomadas de otros trabajos. Por lo que las tablas que se muestran seguidamente, deben ser consideradas como indicadores a considerar en el proyecto, con parámetros de utilidad, en muchos casos indicando valores estimados o deseados a modo de fijar ideas de representación. Con seguridad, tales datos se van a ajustar a los resultados del desarrollo realizado.

Cuadro 3: Estimación de recursos computacionales

Configuración	Nodos	Tiempo estimado
100 estructuras $\times$ 10^4 parámetros (CPU)	1	30 días
100 estructuras $\times$ 10^4 parámetros (HPC+IA)	64	4 horas
1000 estructuras $\times$ 10^5 parámetros (HPC+IA)	256	2 días

En este proceso de estimaciones, se evidencia la ventaja computacional que se tiene en situaciones típicas, similares a la que se propone desarrollar.

3. Estrategia de resolución

3.1. Arquitectura general del sistema

El sistema se organiza en los siguientes módulos:

1. **Preprocesamiento HPC**: filtrado, normalización, división train/validation/test.
2. **Generador de familias de modelos**: crea estructuras paramétricas candidatas.
3. **Predictor GNN**: filtra modelos poco prometedores (85-90 % de ahorro).
4. **Optimizador paralelo**: evalúa modelos prometedores en HPC.
5. **VAE generativo**: sugiere nuevas estructuras basadas en las mejores.
6. **Agente RL**: ajusta hiperparámetros dinámicamente.
7. **Selector de modelo**: elige el mejor por validación cruzada y AIC/BIC.

3.2. Métodos de optimización de parámetros

Cuadro 4: Métodos de optimización según número de parámetros

Parámetros (p)	Método principal	Paralelización
$p \leq 4$	Búsqueda en malla adaptativa	Cada punto de malla $\rightarrow$ un worker
$4 < p \leq 10$	Algoritmos genéticos paralelos	Islas evolutivas, migración de élites
$p > 10$	Optimización bayesiana paralela	Adquisición de lotes con GP

3.3. Balance de carga

Cuadro 5: Estrategias de balance de carga

Recurso	Estrategia
Workers CPU	Cola dinámica de tareas (master-worker) + work stealing
GPUs	Batch processing (simultaneidad de 4096 modelos)
Almacenamiento	I/O asíncrona, checkpointing periódico

3.4. Tolerancia a fallos

- **Checkpointing**: Cada $T = 15$ minutos se guarda estado completo.
- **Replicación**: Modelos IA (GNN, VAE) replicados en 3 nodos.
- **Recuperación automática**: Scheduler detecta workers caídos por heartbeat.
- **Idempotencia**: Las tareas pueden reintentarse sin efectos secundarios.

3.5. Aceleración esperada por IA

Al igual que se ha presentado en la justificación del uso de HPC, en este caso se presentan valores típicos de situaciones similares donde se han aplicado los referidos componentes de IA. El ahorro acumulado, es el considerado a partir de casos exitoso de filtrado temprano de situaciones, que no son conducentes a una solución buscada.

Cuadro 6: Aceleración por componente IA

Componente IA	Factor de aceleración	Ahorro acumulado
Predictor GNN	10× (filtra 90 % evaluaciones)	90 %
VAE generativo	2× (mejor convergencia)	95 %
Agente RL	1.5× (hiperparámetros óptimos)	97 %

4. Propuesta de evaluación experimental

4.1. Métricas de desempeño computacional

En esta etapa se deben considerar las métricas que se mencionan en la siguiente tabla, donde además se han expresado valores esperados (o deseados y representativos), que desde luego, se determinarán aplicados al proyecto desarrollado.

Cuadro 7: Métricas de desempeño computacional

Métrica	Definición	Valor esperado
Speedup	$S_{Up} = T_1/T_{64}$	> 60×
Escalabilidad fuerte	$E = T_1/(N \cdot T_N)$	> 70 % (128 nodos)
Ahorro por predictor GNN	% evaluaciones filtradas	> 85 %
Precisión predictor GNN	Correlación Pearson	> 0,90
Tiempo checkpoint	Tiempo guardado / intervalo	< 5 %

4.2. Métricas de calidad del modelo

Cuadro 8: Métricas de calidad del modelo

Métrica	Definición	Valor deseado
R^2 validación	$1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$	> 0,90
RMSE	$\sqrt{\frac{1}{N} \sum(y - \hat{y})^2}$	< 5 % rango
AIC/BIC	Criterios de información	Minimizado
Estabilidad bootstrap	% términos en > 80 % réplicas	> 80 %

4.3. Experimentos planificados

Cuadro 9: Experimentos planificados

ID	Objetivo	Dataset	Métrica principal
E1	Validar exactitud	Masa-resorte lineal	Error $< 10^{-6}$
E2	Validar IA aceleradora	Masa-resorte + ruido	Ahorro $> 85\%$
E3	Capacidad no lineal	Oscilador de Duffing	Error $< 10^{-4}$
E4	Descubrimiento práctico	Intercambiador de calor	$R^2 > 0,90$
E5	Escalabilidad	Datos sintéticos	Speedup $> 60\times$
E6	Robustez a ruido	SNR variable (0-30 dB)	Degradación $< 50\%$
E7	Ablación IA	Sin predictor GNN	Degradación $> 5\times$ tiempo

5. Resumen de la propuesta

Este proyecto propone un enfoque novedoso y pragmático para el descubrimiento automático de ecuaciones para gemelos digitales, basado en una **arquitectura híbrida que integra**:

1. **Inteligencia Artificial (IA)**: predictor GNN (ej: ahorro 80-90 %), VAE generativo (mutaciones inteligentes), agente RL (adaptación dinámica).
2. **Computación de Altas Prestaciones (HPC)**: simulación paralela, balance de carga, tolerancia a fallos.
3. **Optimización paramétrica**: búsqueda estructurada en familias de modelos.

Las ventajas fundamentales son:

- **Determinismo y reproducibilidad**
- **Escalabilidad** (el problema es embarazosamente paralelo)
- **Aceleración por IA** (ej: reducción de 30 días a 4 horas en 64 nodos)
- **Robustez al ruido** (la estructura predefinida regulariza)
- **Interpretabilidad** (modelos expresados como fórmulas matemáticas)

El enfoque propuesto es viable, innovador y representa una contribución significativa al campo de los Gemelos Digitales, aplicaciones de la Inteligencia Artificial y de la Computación de Altas Prestaciones.

Apéndice A: Glosario de Términos

- Gemelo Digital** Representación virtual de un sistema físico que se actualiza dinámicamente con datos del mundo real, permitiendo simulación, predicción, control y diagnóstico. (Fuente: Grieves, 2014)
- HPC (High Performance Computing)** Computación de altas prestaciones; uso de supercomputadores, clusters o GPUs para resolver problemas que requieren gran capacidad de cómputo, memoria o comunicación.
- IA (Inteligencia Artificial)** Conjunto de técnicas que permiten a las máquinas realizar tareas que requieren inteligencia humana, incluyendo aprendizaje automático, razonamiento y percepción.
- GNN (Graph Neural Network)** Red neuronal de grafos; modelo de aprendizaje profundo diseñado para operar sobre datos estructurados en grafos, donde los nodos y aristas tienen atributos. (Fuente: Wu et al., 2021)
- VAE (Variational Autoencoder)** Autoencoder variacional; modelo generativo que aprende una representación latente probabilística de los datos y puede generar nuevas muestras similares. (Fuente: Kingma y Welling, 2019)
- RL (Reinforcement Learning)** Aprendizaje por refuerzo; paradigma de IA donde un agente aprende a tomar decisiones para maximizar una recompensa acumulada mediante interacción con un entorno. (Fuente: Sutton y Barto, 2018)
- PPO (Proximal Policy Optimization)** Algoritmo de aprendizaje por refuerzo que optimiza políticas de manera estable mediante la limitación de cambios bruscos en la política. (Fuente: Schulman et al., 2017)
- EDO (Ecuación Diferencial Ordinaria)** Ecuación que relaciona una función de una variable con sus derivadas; describe la evolución temporal de sistemas dinámicos.
- EDP (Ecuación en Derivadas Parciales)** Ecuación que involucra derivadas parciales de una función de múltiples variables; utilizada para modelar fenómenos con dependencia espacial.
- Regresión Simbólica** Técnica de aprendizaje automático que busca expresiones matemáticas explícitas que mejor se ajustan a unos datos. (Fuente: Schmidt y Lipson, 2009)
- SINDy (Sparse Identification of Nonlinear Dynamics)** Método para descubrir ecuaciones diferenciales a partir de datos mediante identificación esparsa de términos no lineales. (Fuente: Brunton et al., 2016)
- Optimización paramétrica** Proceso de encontrar los valores óptimos de los parámetros de un modelo para minimizar una función de error.
- Búsqueda en malla (Grid Search)** Método de optimización que evalúa sistemáticamente todas las combinaciones de parámetros en un rango predefinido.
- Algoritmo Genético** Técnica de optimización inspirada en la evolución biológica, que utiliza operadores de selección, cruce y mutación. (Fuente: Holland, 1992)
- Optimización Bayesiana** Método de optimización global que construye un modelo probabilístico de la función objetivo para guiar la búsqueda. (Fuente: Snoek et al., 2012)
- Checkpointing** Técnica de tolerancia a fallos que guarda periódicamente el estado de ejecución de un programa para poder reiniciar desde ese punto en caso de fallo.
- Work stealing** Estrategia de balance de carga donde procesadores inactivos toman tareas de las colas de otros procesadores ocupados.
- Speedup** Medida de ganancia en velocidad al usar múltiples procesadores: $S = T_1/T_N$, donde T_1 es el tiempo secuencial y T_N el tiempo con N procesadores.
- Escalabilidad fuerte** Capacidad de un sistema paralelo para resolver un problema de tamaño fijo más rápidamente al añadir más procesadores.
- Escalabilidad débil** Capacidad de un sistema paralelo para mantener el tiempo de ejecución constante al aumentar el tamaño del problema y el número de procesadores proporcionalmente.

- AIC (Akaike Information Criterion)** Criterio de información para selección de modelos que equilibra la bondad de ajuste con la complejidad del modelo: $AIC = 2p - 2 \ln(L)$. (Fuente: Akaike, 1974)
- BIC (Bayesian Information Criterion)** Criterio de información bayesiano para selección de modelos: $BIC = p \ln(N) - 2 \ln(L)$. (Fuente: Schwarz, 1978)
- SNR (Signal-to-Noise Ratio)** Relación señal-ruido; medida que compara el nivel de una señal deseada con el nivel de ruido de fondo, expresada en decibelios (dB).
- Bootstrap** Técnica estadística que consiste en generar múltiples muestras con reemplazo a partir de los datos originales para estimar la incertidumbre de un estimador. (Fuente: Efron, 1979)
- MSE (Mean Squared Error)** Error cuadrático medio; métrica que calcula el promedio de los cuadrados de los errores entre valores predichos y reales.
- RMSE (Root Mean Squared Error)** Raíz del error cuadrático medio; medida de la magnitud del error de predicción en las mismas unidades que los datos originales.
- MAE (Mean Absolute Error)** Error absoluto medio; promedio de los valores absolutos de los errores entre predicciones y valores reales.
- R² (Coeficiente de determinación)** Medida estadística que indica la proporción de la varianza de la variable dependiente explicada por el modelo.

Referencias

Recursos principales

1. Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716-723.
2. Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15), 3932-3937.
3. Dongarra, J., et al. (2019). Applied mathematics meets high performance computing. *International Journal of High Performance Computing Applications*, 33(5), 834-846.
4. Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1-26.
5. Grieves, M. (2014). Digital twin: Manufacturing excellence through virtual factory replication. *White Paper*, Florida Institute of Technology.
6. Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3rd ed.). MIT Press.
7. Holland, J. H. (1992). Genetic algorithms. *Scientific American*, 267(1), 66-73.
8. Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders. *Foundations and Trends in Machine Learning*, 12(4), 307-392.
9. Mundhenk, T. N., Landajuela, M., Glatt, R., Santiago, C. P., Faissol, D. M., & Petersen, B. K. (2021). Symbolic regression via deep reinforcement learning. *International Conference on Learning Representations (ICLR)*.
10. Petersen, B. K., Landajuela, M., Mundhenk, T. N., Santiago, C. P., Kim, S. K., & Kim, J. T. (2021). Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. *International Conference on Learning Representations (ICLR)*.
11. Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707.
12. Schmidt, M., & Lipson, H. (2009). Distilling free-form natural laws from experimental data. *Science*, 324(5923), 81-85.
13. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint*, arXiv:1707.06347.
14. Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461-464.

15. Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, 25, 2951-2959.
16. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
17. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2021). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1), 4-24.
18. Zhang, Y., Wang, J., & Li, X. (2025). Mathematical expression exploration with graph representation and generative graph neural network. *Neural Networks*, 187, 107405.

Recursos adicionales

- **SRBench:** La Cava, W., et al. (2021). Contemporary symbolic regression methods and their relative performance. *NeurIPS Datasets and Benchmarks Track*. Disponible en: <https://github.com/cavalab/srbench>
- **cp3-bench:** Thing, M. E., & Koksang, S. M. (2024). cp3-bench: A tool for benchmarking symbolic regression algorithms demonstrated with cosmology. *arXiv preprint*, arXiv:2406.15531.
- **PySR:** Cranmer, M. (2023). PySR: Fast parallel symbolic regression in Python. *Journal of Open Source Software*, 8(83), 4884.
- **MPI Standard:** Message Passing Interface Forum (2021). MPI: A Message-Passing Interface Standard Version 4.0. Disponible en: <https://www.mpi-forum.org/docs/>